Information Security
Quiz Refresher: Chapter 2, Answer Sheet

Answers to the 24 recall questions, the 8 statement sets and the 6 exam-style problems

Information Security

2026-10-29

Answers

Round A

A1. Collected, used, shared, retained; understand, influence, avoid. Invisible tracking defeats *understand*; tracking that survives deletion defeats *avoid* (no opt-out defeats *influence*).

A2. Protocol `https`; hostname `informationsecurity.com` (DNS resolves it); port `443`; path `/courses/web-security`; query string `week=5&topic=xss`.

A3. The server keeps nothing once the TCP connection closes: each request is a stranger. A cart or a login needs an identifier carried across requests, so state is added (cookies). Trackers reuse exactly that added state: tracking is the web's own design pointed at another goal.

A4. . . . the page that includes it, not of the server that served it. `analytics.js` executes as `shop.test`'s code: it reads `shop.test`'s non-HttpOnly cookies and `location`, and sets cookies under `shop.test`.

A5. No; it stays in the browser. A script included by the page reads `location.href` (which contains the fragment) and transmits it explicitly.

A6. Copied into bookmarks and messages; written into server logs; sent in the `Referer` header of every request the page triggers, so a third-party image receives it.

A7. A cookie is sent back in a `Cookie` header (1) on every subsequent request *to that domain*, (2) *regardless of which site initiated the request.* `SameSite` qualifies clause (2).

A8. No `Expires`/`Max-Age`: session cookie, deleted when the browser closes; with one: persistent. The server ends a session by deleting (expiring) the cookie.

A9. Default: the current host only. May be set to a parent domain (`Domain=example.com` from `websec.example.com`), covering it and all subdomains. May not name an unrelated domain or a public suffix.

A10. Blocks reading through `document.cookie` (script theft). Does not stop the cookie being *sent*: no effect on CSRF, on tracking, on network theft.

A11. Sent only over HTTPS; settable only from an HTTPS response.

A12. Strict: never attached to cross-site requests. Lax: attached cross-site only for a top-level navigation *and* a safe method (GET, HEAD, OPTIONS, TRACE). None: attached to all cross-site requests, requires `Secure` (Chrome drops it otherwise). Default when absent: Lax.

A13. (1) Is D the cookie's domain or a subdomain when `Domain` was set? Else never. (2) Does the path start with the cookie's path? (3) If `Secure`, is the request HTTPS? (4) Same registrable domain for P and D: attached; cross-site: Strict no, Lax only if top-level and safe, None yes. `HttpOnly` never enters.

A14. eTLD: the longest public suffix the host ends with (`com`, `co.uk`, `github.io`). Registrable domain (eTLD+1): one more label, the unit one owner registered. First-party: destination has the address bar's registrable domain; third-party otherwise.

A15. Origin = (scheme, host, port). SOP governs script *access*: a script reads documents, storage and `document.cookie` of its own origin only. It never prevents the browser from *sending* a cookie. Subdomains: same-site, cross-origin.

A16. (1) An identity in the browser (cookie, storage, or recomputed properties); (2) a channel back (automatic cookie attachment or explicit script send); (3) context (Referer, `location`, an embed parameter). Without (1) every visit is a stranger; without (2) the tracker never hears; without (3) it knows "123 is back" but not where.

A17. A tracker embedded on sites $\mathcal{S}$ with a persistent cookie learns, per browser, the timed sequence of visits to every site in $\mathcal{S}$, with no cooperation from the sites. Each visit triggers a request to the tracker (clause 2, regardless of initiator); it carries the tracker's cookie (clause 1, to that domain), the same value everywhere because it is stored under the tracker's domain; Referer or an embed parameter gives the site.

A18. The script runs as the publisher, so its cookie is stored under the publisher's domain: a different identifier on each site, and the browser never attaches a publisher's cookie to a request for the vendor. The script puts the value into the URL of an explicit request (`track?ga_id=123&site=...`).

A19. No. Blocking applies to cookies set in a third-party *context*; a third-party *script* running in the first-party context still sets first-party cookies. That is the loophole analytics lives in.

A20. Recreating a deleted cookie from a copy held elsewhere (a zombie cookie). `localStorage`: per origin, no expiry, cleared by "clear site data". Flash LSOs: per machine user, across browsers, cleared by none of the browser actions. HTTP cache (`ETag`, `Last-Modified`): per cache, cleared by "clear cache".

A21. (1) The first response to a resource carries `ETag: "<id>"`; (2) the browser caches body and tag; (3) on every later request it sends `If-None-Match: "<id>"`, echoing the identifier even with cookies blocked or in private mode, as long as the cache lives. KissMetrics (August 2011): the same `_km_cid` value was set as a cookie and as the ETag; in private mode with cookies blocked the trace still showed `If-None-Match` with that value.

A22. Two trackers linking the identifiers each assigned to the same browser. The identifier crosses inside the URL of the `302 Location` header (then in the query string of the redirected request), the one place the attachment rule does not police. A script cannot read either cookie across origins (SOP).

A23. SSP: helps publishers offer and sell ad space, forwards each opportunity to the exchange. Ad exchange: runs the auction, distributes the bid request, picks the winner. DSP: evaluates opportunities and bids for its advertisers. The bid request (page URL, approximate location, pseudonymous identifier) is broadcast to *all* DSPs: participating, not winning, is what leaks.

A24. A click goes to the top-most element and `opacity: 0` keeps the frame in hit-testing; the framed page received its cookies (clause 2 of the rule); nothing forbids framing unless the framed page says so. Headers: `X-Frame-Options` (DENY / SAMEORIGIN) and CSP `frame-ancestors`, sent by the *framed* page. Only `frame-ancestors` can list a specific third origin.

Round B

B1. **B only.** (A) false: `HttpOnly` blocks `document.cookie`. (B) true: no attribute means Lax; a link click is a top-level GET. (C) false: POST is not a safe method, Lax refuses. (D) false: an image is not a top-level navigation.

B2. **B and D.** (A) false: different host, different origin. (B) true: same registrable domain `example.com`. (C) false: the scheme differs. (D) true: `Domain=example.com` covers all its subdomains and the request is same-site.

B3. **A and C.** (A) true: the cookie lives under `px.adco.test` and is attached regardless of initiator; None permits cross-site. (B) false: cross-origin, SOP. (C) true: the Referer names the embedding page. (D) false: an embedded image is not top-level, Lax refuses.

B4. **A and D.** (A) true: the script runs as `news.test` and sets the cookie under that domain. (B) false: cookies are attached by destination domain; `news.test`'s cookie never goes to `stats.test`. (C) false: on `shop.test` it finds no cookie under `shop.test` and creates a new one. (D) true: same origin as the page, so it reads `v` and sends it explicitly.

B5. **B and C.** (A) false: different store, different clearing action ("clear site data"). (B) true: the iframe's script runs in `tracker.test`'s origin on every embedding site. (C) true: the tag lives in the HTTP cache. (D) false: the plugin's store was outside the browser profile; no browser action cleared it.

B6. **B and D.** (A) false: tracker 1's cookie goes only to tracker 1; `1234` arrives in the URL. (B) true: the redirected request is to tracker 2's domain. (C) false: SOP, a script reads neither cookie. (D) true: both can now exchange data under either identifier.

B7. **B only.** (A) false: the request is broadcast to every DSP. (B) true: the SSP sells the publisher's ad space. (C) false: the identifier is stable and *enables* recognition. (D) false: only the browser side is observable; what DSPs retain needs an audit.

B8. **B and D.** (A) false: Eve controls that page; the framed page must send the header. (B) true: `'self'` plus one listed origin. (C) false: `visibility: hidden` removes the frame from hit-testing, the click reaches the game. (D) true: the cross-site iframe request carries no Strict cookie, so the portal loads logged out.

Round C

C1. The attachment rule, seven requests

(a) Attached. Step 1: `api.uni.test` is under `uni.test`; step 2: `/app/data` starts with `/app`; step 3: HTTPS; step 4: same site, no cross-site question.

(b) Not attached. Step 2: `/docs` does not start with `/app`.

(c) Attached. Steps 1–3 pass; step 4: cross-site, but a top-level navigation with a safe method (GET), which Lax permits.

(d) Not attached. Step 4: cross-site, top-level, but POST is not a safe method.

(e) Not attached. Step 4: cross-site and not a top-level navigation.

(f) Not attached. Step 3: `Secure` requires HTTPS.

(g) Never. Step 1: the registrable domain is `evil.test`; reading the name from the right, `uni.test` is only a subdomain label.

(h) None: (d) and (e) become attached (None allows every cross-site request); (c) stays attached; (b), (f), (g) unchanged, they failed at steps 2, 3, 1. (i) Strict: (c) becomes not attached (no cross-site request at all); everything else unchanged.

C2. Parties, sites and origins

(a) eTLD `ac.uk`; registrable domain `campus.ac.uk`.

(b) `static.campus.ac.uk`: first (registrable `campus.ac.uk`). `campus.ac.uk.cdn.fastnet.test`: third (owner `fastnet.test`; the left labels are decoration). `campus-ac.uk`: third (registrable `campus-ac.uk`, a different registration). `login.campus.ac.uk:8443`: first (same registrable domain; the port changes the origin, not the site). `campus.co.uk`: third (registrable `campus.co.uk`).

(c) No: different host, so cross-origin; SOP forbids reading. Yes: `Domain=campus.ac.uk` covers both subdomains and the requests are same-site. Access is governed by origin, attachment by site: two different rules.

(d) No: `ac.uk` is a public suffix. Allowing it would let one university set, overwrite or receive cookies of every `*.ac.uk` institution.

(e) Cookie scope is by *name*: the jar is indexed by the host string, so the two names are two domains to the browser, which is why the hosts file maps `lab.test` to the loopback address.

C3. Two trackers, one morning

(a) `px.adco.test`: `u=1`; `news.test`: `v=a`; `shop.test`: `v=b`; `blog.test`: `v=c`.

(b) 09:00 no cookie, set `u=1`, Referer `news.test/politics`; 09:30 `u=1`, Referer `shop.test/cart`. Conclusion: one browser visited both, in that order (the theorem). The `Referer` header gave the site and page.

(c) 09:00 `v=a, site=news.test`; 09:30 `v=b, site=shop.test`; 10:00 `v=c, site=blog.test`. It can count repeat visits within each site (a returning `a` on `news.test`); it cannot tell that `a`, `b` and `c` are one browser.

(d) The e-mail is an external link: `stats.test` now joins `a` and `b` (not `c`) into one profile. Merging first-party identifiers always needs such a key, a login or a fingerprint.

(e) Pixel: 09:00 no cookie, sets `u=1`, but the browser refuses to store a cookie in a third-party context; 09:30 again no cookie, sets `u=2`. Two strangers; the link is gone. Analytics: unchanged, `v` is set by a script running in the first-party context. The blocking targets the context in which a cookie is set, not who wrote the script.

(f) Have `a.js` load a resource from `stats.test` that sets a `stats.test` cookie with `SameSite=None; Secure`: one tracker-domain identifier, attached automatically on every embedding site.

C4. Reading a respawning experiment

(a) Test 2: the cookie returned while L and E still held 4411: consistent with restoration from `localStorage` (or from the ETag); not proved, because either store, or a fingerprint, could explain it. Test 3: storage was wiped, yet C is 4411 and E still echoed 4411: consistent with ETag restoration; a fingerprint remains an alternative explanation. Test 4: all stores cleared and every identifier is new (8820), so the stores *were* cleared; P stayed 6: consistent with stateless relinking through F.

(b) The action cleared storage before the visit. During the visit the cookie was recreated from the ETag (4411), and the page's script then rewrote 4411 into `localStorage`: L is refilled, not surviving.

(c) No stateful store survived (all three identifiers changed), so the only remaining explanation is an identifier recomputed from the browser's properties: F = 17 was the same before and after. Still "consistent with": the experiment does not show the tracker *used* F; it shows that nothing else could have.

(d) Test 5: fresh stores (9931) but same F and same P: F is computed from the browser, not

from any store, and it survives private mode. Test 6: a different browser gives F = 29 and a new profile 11: F identifies a *browser*, not a person or a machine. False positive: two different users with identical browser setups share an F and would be merged into one profile.

(e) Clear cookies, site data and the cache *together* (test 4's action). "Delete cookies" alone leaves `localStorage` and the ETag (test 2); "clear site data" leaves the HTTP cache (test 3). What survives even test 4 is stateless and needs the defences of Chapter 3.

C5. Write the sync

(a)

1. `GET https://ad-x.test/pixel.gif`, `Cookie: x=X9`, `Referer: https://news.test/`.
2. `302 Found`, `Location: https://ad-y.test/sync?x=X9`.
3. `GET https://ad-y.test/sync?x=X9`, `Cookie: y=Y4`, `Referer: https://news.test/` (the browser follows automatically).
4. `ad-y.test` stores `Y4` ↔ `X9`; optional return: `302 Location: https://ad-x.test/sync?y=Y4`.
5. `GET https://ad-x.test/sync?y=Y4`, `Cookie: x=X9`: `ad-x.test` stores `X9` ↔ `Y4`. Final response: a 1-pixel image.

(b) Before: `ad-x.test` knows the browser as X9 with the visit log of its own sites; `ad-y.test` knows it as Y4 with its own log; neither can relate them. After: both hold the pair and can exchange logs and attributes under either identifier; each one's coverage becomes the union of the two sets of sites.

(c) A script on `news.test` runs in `news.test`'s origin and can read neither `ad-x.test`'s nor `ad-y.test`'s cookie (SOP). Only a request *to* `ad-y.test` carries `y`; the redirect makes the browser issue such a request while the URL carries X9.

(d) Clause 1, "to that domain": the redirected request is destined for `ad-y.test`, so its jar entry is attached. X9 crosses in the `Location` header of the 302, then sits in the query string of request 3: inside the URL, which the rule does not police.

(e) Yes: on `news.test` both trackers have cookies in that site's jar, and the redirect links them there. The partition prevents the *reuse* of X9 across publishers: on `shop.test` the jar holds a different `ad-x.test` cookie, so the morning's visits are no longer one identifier.

(f) Between an SSP and a DSP. The DSP then recognises in each bid request (carrying the SSP's identifier) a user it has already profiled under its own identifier, and bids accordingly.

C6. Framing the portal

(a) (1) The click goes to the top-most element and an `opacity: 0` frame stays in hit-testing: `visibility: hidden` or `pointer-events: none` would route the click to the game. (2) The framed portal received its session cookie (clause 2 of the attachment rule): a Lax or Strict session cookie would load it logged out. (3) Nothing forbids framing unless the portal says so: a framing header on the portal's response would stop the frame from loading.

(b) `Content-Security-Policy: frame-ancestors 'none'` plus `X-Frame-Options: DENY` for browsers that predate CSP. Both on the portal's responses.

(c) `Content-Security-Policy: frame-ancestors 'self' https://lms.uni.test`. `X-Frame-Options` knows only DENY and SAMEORIGIN (`ALLOW-FROM` is obsolete); it

cannot list a third origin, and `frame-ancestors` takes precedence anyway.

(d) For the TP attack, yes: the iframe is a cross-site, non-top-level request, so under Lax the session cookie is not attached, the framed portal is logged out, and the hijacked click confirms nothing. Not recommended as *the* defence: the page is still embeddable, any action that needs no session can still be hijacked, and the protection depends on a cookie attribute rather than on a statement by the page. `frame-ancestors` is the control.

(e) Frame-busting runs inside the framed page and can be neutralised by the framing page (sandboxed iframes with scripts disabled, `onbeforeunload` tricks, double framing). A header is enforced by the browser before any script runs and cannot be overridden by the embedder.